Abstract

Paper title

UIBenchKit: A Unified Toolkit for Design-to-Code Model Evaluation

Accepted paper: https://arxiv.org/pdf/2605.13141

Purpose

UIBenchKit provides a unified execution and evaluation workflow for automated design-to-code research. It accepts benchmark or custom screenshot inputs, routes them through five generation methods and multiple multimodal model providers, renders generated HTML with Playwright, computes structural and visual metrics, records cost metadata, and exposes results through a CLI, REST API, and analytical web interface.

The artifact packages the Python backend and CLI with the React/Node web application as two Linux containers. Its reduced smoke test submits one bundled screenshot to GPT-4o using direct prompting and verifies generation, rendering, CLIP evaluation, reporting, artifact download, GUI proxying, and browser rendering. Published large-scale results remain archived on Hugging Face.

Badge

We apply for **Artifacts Available** because the complete source snapshots, prebuilt container images, checksums, documentation, and version metadata are deposited in a permanent archival repository with a DOI. Public GHCR images are also provided as a convenient mirror.

We apply for **Artifacts Reusable** because the artifact has a documented under-30-minute setup, automated success checks, pinned source and dependency versions, separate backend/CLI/GUI interfaces, extension documentation, and instructions for both reduced and full workflows.

Technology skills assumed by the reviewer evaluating the artifact and hardware requirements

Reviewers need basic Bash and Docker Compose skills; no knowledge of the implementation languages is required. Required tools are Docker Engine 24 or newer, Docker Compose v2.20 or newer, Bash, `curl`, `gzip`, `tar`, and GNU `sha256sum` (or `shasum` on macOS). The packaged architecture is Linux AMD64. Docker Desktop is supported on Windows through WSL2 and on macOS when configured for Linux containers.

A minimum of 2 CPU cores, 8 GB RAM, and 25 GB free disk is required. No GPU is required for installation or the reduced workflow. Rebuilding images benefits from 4 CPU cores, 16 GB RAM, and 60 GB free disk. The real-model smoke test also requires internet access and an author-supplied, rate-limited OpenAI API key with GPT-4o access; the installation check is key-free.

Provenance

Archival artifact: https://doi.org/10.5281/zenodo.21175610. Source repository and GHCR instructions: https://github.com/chinh02/uibenchkit-artifact. Raw benchmark artifacts: https://huggingface.co/datasets/chinh02/UIBenchKit. Project website: https://www.uibenchkit.com/.

Instructions

1. Download and verify. Download the source archive, two image archives, artifact abstract, and `SHA256SUMS` from the archival DOI into one directory. Run `sha256sum --check SHA256SUMS` and require every entry to report `OK`.

2. Load and extract. Load each compressed image with `gzip --decompress --stdout <image>.tar.gz | docker load`. Extract the source with `tar --extract --gzip --file uibenchkit-artifact-v1.0.0-ase2026-source.tar.gz`, then enter the extracted directory. The README gives the two exact image filenames and macOS checksum alternative.

3. Check installation. If ports 5000 or 3000 are occupied, export the documented `BACKEND_PORT` and `GUI_PORT` overrides. Run `bash ./scripts/health-check.sh`. This key-free check starts both services and validates the backend, CLI, GUI, public leaderboard, and browser routes. It finishes with `UIBenchKit services are healthy.`

4. Run the reduced workflow. Put the confidentially supplied OpenAI key in an untracked `.env`, export it as shown in the README, and run `bash ./scripts/smoke.sh`. Success ends with `SMOKE TEST PASSED`, a run ID, and a finite CLIP score. This normally completes in under 30 minutes.

Dataset schema and usage. Published runs are stored under `raw-data/<dataset>_<method>_<model>_<timestamp>/`. Each run contains `evaluation.json`, `run_metadata.json`, `results.json`, `cost_report.json`, generated `<sample_id>.html`, and rendered `<sample_id>.png` files. The schema supports auditing individual generations, comparing methods and models, and regenerating the CSV/JSON leaderboard with `summarize_leaderboard.py`. Reviewers can inspect archived Design2Code and DCGen runs or use the bundled custom PNG to exercise the dataset-independent smoke path. The README documents cleanup, claim coverage, and the optional long-running workflow.